

Documentación de patrones de diseño y principios SOLID utilizados en el proyecto

App Películas por Ver

Marely Jacome Hernandez

8 de mayo de 2026

1. Introducción

En este documento explico los patrones de diseño y principios SOLID que utilicé en mi proyecto *App Películas por Ver*. La aplicación permite registrar películas, subir portadas, marcar si una película ya fue vista, agregar un comentario personal y seleccionar diferentes reacciones.

El proyecto está dividido en dos partes principales: el frontend, desarrollado con Flutter, y la API, desarrollada con Spring Boot. Como estudiante principiante, entendí que los patrones y SOLID ayudan a que el código esté más ordenado, sea más fácil de entender y pueda modificarse con menos problemas.

2. Resumen general de patrones utilizados

Tabla 1: Patrones y componentes arquitectónicos utilizados en el proyecto

Patrón	Dónde se usó	Propósito en el proyecto
Repository	Frontend y API	Separar el acceso a datos de la interfaz y de la lógica principal.
Service Layer	Frontend y API	Organizar la lógica de comunicación y reglas del sistema.
Facade	Frontend	Simplificar el acceso de las pantallas a servicios, repositorios y autenticación.
Factory Method	Frontend y API	Centralizar la creación de objetos, respuestas y elementos visuales.
Strategy	Frontend	Organizar los filtros de películas en clases separadas.
Adapter	Frontend	Convertir imágenes seleccionadas al formato que necesita la API.
Singleton	Frontend	Mantener una única instancia para manejar la sesión del usuario.
DTO / Model	Frontend y API	Transportar datos entre capas sin exponer directamente toda la estructura interna.
MVC	API	Separar modelos, controladores y lógica del backend.
Security Filter	API	Validar el token de autenticación antes de permitir acceso a rutas protegidas.

3. Principios SOLID aplicados en el proyecto

Además de los patrones de diseño, en el proyecto también se aplicaron algunos principios SOLID. SOLID es un conjunto de principios que ayudan a que el código sea más ordenado, fácil de mantener y menos dependiente entre clases.

Como estudiante principiante, entiendo SOLID como una guía para que cada parte del sistema tenga una responsabilidad clara y para evitar que una clase haga demasiadas cosas al mismo tiempo.

Es importante aclarar que SOLID no es un patrón de diseño, sino un conjunto de principios. Por esa razón, en este documento lo separe de la tabla de patrones utilizados.

Tabla 2: Principios SOLID aplicados en el proyecto

Principio	Dónde se aplicó	Propósito en el proyecto
Single Responsibility Principle	Frontend y API	Separar responsabilidades para que cada clase tenga una función principal.
Open/Closed Principle	Frontend	Permitir agregar nuevos filtros o nuevas reacciones sin modificar demasiado el código existente.
Liskov Substitution Principle	Frontend	Permitir que las implementaciones concretas puedan usarse donde se espera una abstracción.
Interface Segregation Principle	Frontend	Evitar que las pantallas dependan directamente de clases grandes o de detalles internos.
Dependency Inversion Principle	Frontend y API	Hacer que las clases dependan de abstracciones o dependencias inyectadas, no de objetos creados directamente dentro de ellas.

3.1. Single Responsibility Principle

3.1.1. Cómo se usa de manera general

El principio de responsabilidad única, conocido como *Single Responsibility Principle*, indica que una clase debe tener una sola responsabilidad principal.

Yo lo entiendo como que una clase no debería encargarse de todo. Si una clase muestra información, no debería también conectarse directamente a la API, validar tokens, guardar datos y manejar toda la lógica de negocio.

3.1.2. Cómo lo usé en mi proyecto

En el frontend lo apliqué separando el proyecto en diferentes capas y archivos. Por ejemplo:

```
screens
widgets
services
repositories
models
core/facades
core/network
core/strategies
```

Cada carpeta tiene una responsabilidad diferente. Las pantallas muestran la información, los servicios se comunican con la API, los repositorios separan el acceso a datos, los modelos representan información y las estrategias ayudan a organizar los filtros.

También se aplicó en la API, porque el backend está dividido en:

```
controllers
models
repository
service
security
payload
```

Por ejemplo, `PeliculaController.java` recibe peticiones relacionadas con películas, pero no debería encargarse de toda la lógica interna. Esa lógica se delega a servicios o repositorios.

3.1.3. Para qué lo usé

Lo usé para que el código fuera más fácil de entender. Si necesito modificar algo de las reacciones, puedo buscar en las clases relacionadas con reacciones. Si necesito modificar algo de autenticación, puedo buscar en los archivos de autenticación.

Esto evita que todo el código esté mezclado en una sola clase.

3.2. Open/Closed Principle

3.2.1. Cómo se usa de manera general

El principio abierto/cerrado, conocido como *Open/Closed Principle*, indica que una clase debería estar abierta para extenderse, pero cerrada para modificarse.

Yo lo entiendo como que debería poder agregar nuevas funciones sin tener que cambiar demasiado el código que ya funciona.

3.2.2. Cómo lo usé en mi proyecto

En el frontend se aplicó principalmente con el patrón Strategy en los filtros de películas.

El archivo:

```
lib/core/strategies/movie_filter_strategy.dart
```

contiene diferentes estrategias como:

```
AllMoviesStrategy  
WatchedMoviesStrategy  
PendingMoviesStrategy  
SearchMoviesStrategy
```

Gracias a esto, si en el futuro quisiera agregar otro filtro, por ejemplo “películas mejor calificadas”, podría crear una nueva estrategia sin modificar toda la pantalla principal.

También se puede observar en las reacciones. Las reacciones tienen una fábrica visual en:

```
lib/core/factories/reaccion_ui_factory.dart
```

Si quisiera cambiar el ícono o color de una reacción, lo puedo hacer desde esa fábrica sin modificar todas las pantallas donde se muestran reacciones.

3.2.3. Para qué lo usé

Lo usé para que el proyecto pudiera crecer sin romper partes que ya estaban funcionando. Esto es útil porque la app puede seguir agregando nuevas opciones, filtros o elementos visuales de forma más ordenada.

3.3. Liskov Substitution Principle

3.3.1. Cómo se usa de manera general

El principio de sustitución de Liskov indica que una clase hija o implementación concreta debe poder usarse en lugar de su clase base o abstracción sin que el programa falle.

Yo lo entiendo como que si una clase promete cumplir cierto comportamiento, cualquier implementación debe respetar ese comportamiento.

3.3.2. Cómo lo usé en mi proyecto

En el frontend se aplicó con el repositorio de películas.

Existe una abstracción llamada:

```
PeliculaRepository
```

y una implementación concreta llamada:

```
RemotePeliculaRepository
```

La interfaz trabaja con `PeliculaRepository`, pero realmente puede usar `RemotePeliculaRepository` porque esta implementación cumple con los métodos esperados.

También se puede observar en las estrategias de filtrado. Todas las estrategias implementan la misma estructura de `MovieFilterStrategy`. Por eso la pantalla puede trabajar con una estrategia sin importar si es para obtener todas las películas, las vistas, las pendientes o buscar por nombre.

3.3.3. Para qué lo usé

Lo usé para que el código pudiera trabajar con abstracciones. Esto permite que en el futuro se pueda cambiar una implementación por otra sin modificar toda la aplicación.

Por ejemplo, si algún día quisiera usar una base de datos local en lugar de una API remota, podría crear otra implementación del repositorio respetando la misma estructura.

3.4. Interface Segregation Principle

3.4.1. Cómo se usa de manera general

El principio de segregación de interfaces indica que una clase no debería depender de métodos que no necesita.

Yo lo entiendo como que es mejor tener contratos claros y útiles, en lugar de obligar a una clase a conocer demasiadas cosas que no usa.

3.4.2. Cómo lo usé en mi proyecto

En mi proyecto este principio se aplicó de forma parcial, principalmente al evitar que las pantallas usen directamente todos los servicios internos.

Por ejemplo, las pantallas no dependen directamente de `PeliculaService`, `AuthService`, `ApiClient` o `SessionManager`. En lugar de eso, usan `MovieAppFacade`, que les ofrece solamente las operaciones necesarias para la interfaz.

También se observa en el uso de modelos y DTOs, porque no siempre se envía o recibe toda la entidad completa. Por ejemplo, para actualizar un comentario personal se usa un objeto específico con el comentario, en lugar de obligar a enviar todos los datos de la película.

3.4.3. Para qué lo usé

Lo usé para que las pantallas dependieran de operaciones más claras y no de todos los detalles internos del sistema.

Aunque este principio no está aplicado de forma perfecta en todo el proyecto, sí ayudó a reducir dependencias innecesarias entre la interfaz y las clases internas.

3.5. Dependency Inversion Principle

3.5.1. Cómo se usa de manera general

El principio de inversión de dependencias indica que las clases de alto nivel no deberían depender directamente de clases concretas de bajo nivel, sino de abstracciones.

No es lo mismo que inyección de dependencias. La inversión de dependencias es un principio de SOLID, mientras que la inyección de dependencias es una técnica que ayuda a aplicarlo.

3.5.2. Cómo lo usé en mi proyecto

En el frontend se aplicó cuando `MovieAppFacade` trabaja con la abstracción `PeliculaRepository`, en lugar de depender directamente de una implementación concreta.

Esto permite que la fachada no tenga que saber exactamente cómo se obtienen los datos. Solo sabe que existe un repositorio que cumple con ciertos métodos.

En la API Spring Boot también se aplicó mediante la inyección de dependencias por constructor. Por ejemplo, los controladores y servicios reciben objetos como:

```
PeliculaRepository  
PeliculaReaccionRepository  
CloudinaryService  
UserRepository
```

Spring Boot se encarga de inyectar esas dependencias automáticamente, por lo que las clases no tienen que crearlas manualmente con `new`.

3.5.3. Para qué lo usé

Lo usé para reducir el acoplamiento entre clases. Esto significa que una clase no depende tanto de cómo se construye otra clase, sino que recibe lo que necesita desde afuera.

Esto hace que el proyecto sea más fácil de mantener y modificar, porque si cambia una implementación, no necesariamente se tiene que cambiar toda la aplicación.

4. Repository

4.1. Cómo se usa de manera general

El patrón Repository, o patrón Repositorio, se utiliza para separar la forma en que se obtienen los datos del resto de la aplicación. En lugar de que una pantalla o controlador consulte directamente una base de datos o una API, se crea una capa intermedia llamada repositorio.

Yo lo entiendo como una especie de intermediario entre la aplicación y la fuente de datos. La aplicación pide la información al repositorio, y el repositorio se encarga de obtenerla.

4.2. Cómo lo usé en mi proyecto

En el frontend lo utilicé en el archivo:

```
lib/repositories/pelicula_repository.dart
```

Ahí se creó una abstracción llamada `PeliculaRepository`, donde se definen operaciones como obtener películas, crear películas, editar películas, eliminar películas, agregar reacciones y obtener las estadísticas de “Mis reacciones”.

```

/// Pattern: Repository
/// Desacopla la UI de la fuente real de datos. Permite intercambiar implementación.
abstract class PeliculaRepository {
  Future<List<Pelicula>> obtenerPeliculas();

  Future<List<Pelicula>> buscarPorNombre(String nombre);

  Future<List<Pelicula>> filtrarPorVista(bool vista);

  Future<Pelicula> crearPelicula({
    required String nombre,
    required String genero,
    required double calificacion,
    required bool vista,
    required XFile imagen,
  });

  Future<Pelicula> editarPelicula({
    required int id,
    required String nombre,
    required String genero,
    required double calificacion,
    required bool vista,
    XFile? imagen,
  });

  Future<void> eliminarPelicula(int id);

  Future<Pelicula> cambiarEstadoVista(int id, bool vista);

  Future<Pelicula> actualizarComentarioPersonal({
    required int id,
    required String comentarioPersonal,
  });

  Future<List<ReaccionPelicula>> obtenerReaccionesDePelicula(int id);

  Future<List<ReaccionPelicula>> agregarReaccion({
    required int id,
    required TipoReaccion tipoReaccion,
  });

  Future<List<ReaccionPelicula>> eliminarReaccion({
    required int id,
    required TipoReaccion tipoReaccion,
  });

  Future<List<ResumenReaccion>> obtenerMisReacciones();

  Future<List<Pelicula>> obtenerPeliculasPorReaccion(
    TipoReaccion tipoReaccion,
  );
}

```

Figura 1: Implementación del patrón Repository en el archivo `pelicula_repository.dart`.

4.3. Para qué lo usé

Lo usé para que la interfaz no tuviera que saber directamente cómo se obtienen los datos. Por ejemplo, la pantalla principal no necesita saber cómo se hace una petición HTTP; solamente pide las películas al repositorio.

Esto ayuda a que el código esté más ordenado y sea más fácil cambiar la forma de obtener datos en el futuro.

5. Service Layer

5.1. Cómo se usa de manera general

El patrón Service Layer, o Capa de Servicio, sirve para concentrar la lógica principal de la aplicación en una capa separada. La idea es que los controladores o pantallas no tengan toda la lógica mezclada.

En otras palabras, el servicio se encarga de decidir qué hacer, mientras que la pantalla o el controlador solo llaman al servicio.

5.2. Cómo lo usé en mi proyecto

En el frontend lo utilicé en:

```
lib/services/pelicula_service.dart  
lib/services/auth_service.dart
```

El archivo `pelicula_service.dart` se encarga de comunicarse con la API para obtener películas, crear películas, editar películas, guardar comentarios personales y manejar reacciones.

El archivo `auth_service.dart` se encarga del inicio de sesión, registro y manejo del token del usuario.

```

/// PATRON: Service Layer
/// Centraliza la comunicación con la API para que la interfaz no haga
/// peticiones HTTP directamente.
class PeliculaService {
  PeliculaService({
    ApiClient? apiClient,
    MultipartImageAdapter? imageAdapter,
  }) : _apiClient = apiClient ?? ApiClient(),
       _imageAdapter = imageAdapter ?? const MultipartImageAdapter();

  final ApiClient _apiClient;
  final MultipartImageAdapter _imageAdapter;

  Uri _buildUri(String path, [Map<String, dynamic>? query]) {
    return _apiClient.buildUri(path, query);
  }

  Future<List<Pelicula>> obtenerPeliculas() async {
    final response = await _apiClient.get('/api/peliculas');
    final List<dynamic> data = jsonDecode(response.body);
    return data.map((e) => Pelicula.fromJson(e)).toList();
  }

  Future<List<Pelicula>> buscarPorNombre(String nombre) async {
    final response = await _apiClient.get(
      '/api/peliculas/buscar',
      query: {'nombre': nombre},
    );
    final List<dynamic> data = jsonDecode(response.body);
    return data.map((e) => Pelicula.fromJson(e)).toList();
  }

  Future<List<Pelicula>> filtrarPorVista(bool vista) async {
    final response = await _apiClient.get(
      '/api/peliculas/filtrar',
      query: {'vista': vista},
    );
    final List<dynamic> data = jsonDecode(response.body);
    return data.map((e) => Pelicula.fromJson(e)).toList();
  }

  Future<Pelicula> crearPelicula({
    required String nombre,
    required String genero,
    required double calificacion,
    required bool vista,
    required XFile imagen,
  }) async {
    final request = http.MultipartRequest(
      'POST',
      _buildUri('/api/peliculas'),
    );
  }
}

```

Figura 2: Implementación de la capa de servicio en `pelicula_service.dart`.

5.3. Para qué lo usé

Lo usé para separar la lógica de comunicación con la API de las pantallas. Así, la interfaz no tiene que conocer todos los detalles de las peticiones HTTP ni de cómo se procesan las respuestas.

6. Facade Pattern

6.1. Cómo se usa de manera general

El patrón Facade, o Fachada, sirve para simplificar el acceso a varias clases o servicios. En vez de que una pantalla tenga que comunicarse con muchas clases diferentes, se crea una fachada que centraliza esas operaciones.

Yo lo entiendo como una entrada principal para usar varias partes del sistema sin tener que conocer todos los detalles internos.

6.2. Cómo lo usé en mi proyecto

En el frontend lo utilicé en:

```
lib/core/facades/movie_app_facade.dart
```

La clase `MovieAppFacade` centraliza operaciones relacionadas con autenticación, películas, comentarios personales y reacciones.

```

/// PATRÓN: Facade
/// Entrada única para autenticación, películas y reacciones. Desacopla UI de servicios.
class MovieAppFacade {
  MovieAppFacade({
    required AuthService authService,
    required PeliculaRepository peliculaRepository,
  }) : _authService = authService,
      _peliculaRepository = peliculaRepository;

  final AuthService _authService;
  final PeliculaRepository _peliculaRepository;

  bool get estaAutenticado => _authService.estaAutenticado;

  String? get token => _authService.token;

  Future<void> iniciarSesion({
    required String username,
    required String password,
  }) {
    return _authService.login(
      username: username,
      password: password,
    );
  }

  Future<void> registrarUsuario({
    required String username,
    required String email,
    required String password,
  }) {
    return _authService.register(
      username: username,
      email: email,
      password: password,
    );
  }

  void cerrarSesion() {
    _authService.logout();
  }

  Future<List<Pelicula>> obtenerPeliculas() {
    return _peliculaRepository.obtenerPeliculas();
  }

  Future<List<Pelicula>> buscarPorNombre(String nombre) {
    return _peliculaRepository.buscarPorNombre(nombre);
  }

  Future<List<Pelicula>> filtrarPorVista(bool vista) {
    return _peliculaRepository.filtrarPorVista(vista);
  }
}

```

Figura 3: Fachada principal del frontend mediante la clase `MovieAppFacade`.

También se aplicó una fachada técnica en:

```
lib/core/network/api_client.dart
```

La clase `ApiClient` centraliza las peticiones HTTP, los encabezados, el token de autenticación y la validación de respuestas.

```

/// PATRÓN: Facade (Cliente HTTP)
/// Centraliza URIs, headers (auth) y validación de respuestas HTTP.
class ApiClient {
    ApiClient({
        SessionManager? sessionManager,
        String? baseUrl,
    }) : _sessionManager = sessionManager ?? SessionManager(),
        _baseUrl = baseUrl ?? ApiConfig.baseUrl;

    final SessionManager _sessionManager;
    final String _baseUrl;

    Uri buildUri(String path, [Map<String, dynamic>? query]) {
        final uri = Uri.parse('${_baseUrl}$path');

        if (query == null || query.isEmpty) {
            return uri;
        }

        return uri.replace(
            queryParameters: query.map(
                (key, value) => MapEntry(key, value.toString()),
            ),
        );
    }

    Map<String, String> headers({bool withAuth = true}) {
        final headers = <String, String>{
            'Content-Type': 'application/json',
        };

        if (withAuth && _sessionManager.token != null) {
            headers['Authorization'] = 'Bearer ${_sessionManager.token}';
        }

        return headers;
    }

    Map<String, String> authHeaders() {
        final headers = <String, String>{};

        if (_sessionManager.token != null) {
            headers['Authorization'] = 'Bearer ${_sessionManager.token}';
        }

        return headers;
    }

    Future<http.Response> get(
        String path, {
        Map<String, dynamic>? query,
        bool withAuth = true,
    }) {

```

Figura 4: Fachada técnica para centralizar las peticiones HTTP mediante `ApiClient`.

6.3. Para qué lo usé

Lo usé para que las pantallas no llamen directamente a muchas clases. Por ejemplo, desde una pantalla puedo usar:

```

facade.obtenerPeliculas()
facade.agregarReaccion()
facade.actualizarComentarioPersonal()

```

sin preocuparme por cómo se conecta internamente con la API.

Esto hace que el código de las pantallas sea más limpio.

7. Factory Method

7.1. Cómo se usa de manera general

El patrón Factory Method, o Método Fábrica, se usa para centralizar la creación de objetos. En lugar de crear objetos manualmente en varias partes del código, se crea un método encargado de construirlos.

Esto ayuda a evitar código repetido.

7.2. Cómo lo usé en mi proyecto

En el frontend lo usé en los modelos:

```
lib/models/pelicula.dart  
lib/models/reaccion_pelicula.dart  
lib/models/resumen_reaccion.dart
```

Por ejemplo, `Pelicula.fromJson()` sirve para crear un objeto `Pelicula` a partir de la respuesta JSON que llega desde la API.


```

/// Patrón creacional: Factory Method.
/// Centraliza la creación de objetos Pelicula desde JSON.
/// Así la conversión de datos no se repite en servicios o pantallas.
factory Pelicula.fromJson(Map<String, dynamic> json) {
  return Pelicula(
    id: json['id'] is int
      ? json['id'] as int
      : int.tryParse((json['id'] ?? '').toString()),
    nombre: (json['nombre'] ?? '').toString(),
    portada: (json['portada'] ?? '').toString(),
    genero: (json['genero'] ?? '').toString(),
    calificacion: (json['calificacion'] as num?)?.toDouble() ?? 0.0,
    vista: json['vista'] as bool? ?? json['visto'] as bool? ?? false,
    comentarioPersonal: (json['comentarioPersonal'] ?? '').toString(),
  );
}

Map<String, dynamic> toJson() {
  return {
    'id': id,
    'nombre': nombre,
    'portada': portada,
    'genero': genero,
    'calificacion': calificacion,
    'vista': vista,
    'comentarioPersonal': comentarioPersonal,
  };
}

Pelicula copyWith({
  int? id,
  String? nombre,
  String? portada,
  String? genero,
  double? calificacion,
  bool? vista,
  String? comentarioPersonal,
}) {
  return Pelicula(
    id: id ?? this.id,
    nombre: nombre ?? this.nombre,
    portada: portada ?? this.portada,
    genero: genero ?? this.genero,
    calificacion: calificacion ?? this.calificacion,
    vista: vista ?? this.vista,
    comentarioPersonal: comentarioPersonal ?? this.comentarioPersonal,
  );
}
}

```

Figura 5: Método de creación de objetos en el modelo Pelicula.

```

7  /// PATRÓN: DTO / Factory Method
8  /// Representa reacción de API. fromJson centraliza creación desde JSON.
9  class ReaccionPelicula {
10   final TipoReaccion tipoReaccion;
11   final String nombre;
12
13   const ReaccionPelicula({
14     required this.tipoReaccion,
15     required this.nombre,
16   });
17
18   factory ReaccionPelicula.fromJson(Map<String, dynamic> json) {
19     final tipoApi = (json['tipoReaccion'] ?? '').toString();
20
21     return ReaccionPelicula(
22       tipoReaccion: TipoReaccionExtension.fromApiValue(tipoApi),
23       nombre: (json['nombre'] ?? '').toString(),
24     );
25   }
26 }

```

Figura 6: Método de creación de objetos en el modelo ReaccionPelicula.

```

peliculas_front / lib / models / resumen_reaccion.dart
import 'reaccion_pelicula.dart';

/// PATRÓN: DTO / Factory Method
/// Resumen estadístico de reacción. fromJson convierte datos de API.
class ResumenReaccion {
  final TipoReaccion tipoReaccion;
  final String nombre;
  final int total;

  const ResumenReaccion({
    required this.tipoReaccion,
    required this.nombre,
    required this.total,
  });

  factory ResumenReaccion.fromJson(Map<String, dynamic> json) {
    final tipoApi = (json['tipoReaccion'] ?? '').toString();

    return ResumenReaccion(
      tipoReaccion: TipoReaccionExtension.fromApiValue(tipoApi),
      nombre: (json['nombre'] ?? '').toString(),
      total: (json['total'] as num?).toInt() ?? 0,
    );
  }
}

```

Figura 7: Método de creación de objetos en el modelo ResumenReaccion.

También se relaciona con la creación de elementos visuales para las reacciones mediante:

```
lib/core/factories/reaccion_ui_factory.dart
```

7.3. Para qué lo usé

Lo usé para no repetir la lógica de creación de objetos o elementos visuales.

Por ejemplo, si quiero cambiar el ícono de “Me gusta”, solo tendría que modificarlo en `ReaccionUiFactory`, no en varias pantallas.

8. Strategy Pattern

8.1. Cómo se usa de manera general

El patrón Strategy, o Estrategia, permite tener diferentes formas de realizar una misma acción. Cada forma se separa en una clase distinta.

Sirve para evitar que una clase tenga demasiados condicionales como `if` o `switch`.

8.2. Cómo lo usé en mi proyecto

En el frontend lo usé en:

```
lib/core/strategies/movie_filter_strategy.dart
```

Ahí se crearon diferentes estrategias para filtrar películas:

```
AllMoviesStrategy  
WatchedMoviesStrategy  
PendingMoviesStrategy  
SearchMoviesStrategy
```

Cada una representa una forma distinta de consultar películas.

```

/// PATRÓN: Strategy
/// Encapsula criterios de filtrado. Evita condicionales complejos en la UI.
abstract class MovieFilterStrategy {
    Future<List<Película>> obtenerPelículas(MovieAppFacade facade);
}

/// Estrategia para obtener todas las películas.
class AllMoviesStrategy implements MovieFilterStrategy {
    @override
    Future<List<Película>> obtenerPelículas(MovieAppFacade facade) {
        return facade.obtenerPelículas();
    }
}

/// Estrategia para obtener solo películas vistas.
class WatchedMoviesStrategy implements MovieFilterStrategy {
    @override
    Future<List<Película>> obtenerPelículas(MovieAppFacade facade) {
        return facade.filtrarPorVista(true);
    }
}

/// Estrategia para obtener solo películas pendientes.
class PendingMoviesStrategy implements MovieFilterStrategy {
    @override
    Future<List<Película>> obtenerPelículas(MovieAppFacade facade) {
        return facade.filtrarPorVista(false);
    }
}

/// Estrategia para buscar películas por nombre.
class SearchMoviesStrategy implements MovieFilterStrategy {
    SearchMoviesStrategy(this.nombre);

    final String nombre;

    @override
    Future<List<Película>> obtenerPelículas(MovieAppFacade facade) {
        return facade.buscarPorNombre(nombre);
    }
}

```

Figura 8: Implementación del patrón Strategy para organizar los filtros de películas.

8.3. Para qué lo usé

Lo usé para organizar mejor los filtros de la pantalla principal.

Por ejemplo, una estrategia obtiene todas las películas, otra obtiene solo las vistas, otra obtiene las pendientes y otra busca por nombre.

Esto ayuda a que la pantalla principal no tenga toda la lógica de filtrado mezclada.

9. Adapter Pattern

9.1. Cómo se usa de manera general

El patrón Adapter, o Adaptador, se usa cuando una parte del sistema tiene un formato de datos, pero otra parte necesita un formato diferente.

El adaptador se encarga de convertir un formato en otro.

9.2. Cómo lo usé en mi proyecto

En el frontend lo usé en:

```
lib/core/adapters/multipart_image_adapter.dart
```

La aplicación selecciona imágenes como objetos `XFile`, pero para enviarlas a la API se necesita convertirlas a `MultipartFile`.

Entonces el adaptador hace esta conversión:

```
XFile -> MultipartFile
```

```
import 'package:http/http.dart' as http;
import 'package:image_picker/image_picker.dart';

/// PATRÓN: Adapter
/// Convierte XFile (ImagePicker) a MultipartFile para subida HTTP.
class MultipartImageAdapter {
  const MultipartImageAdapter();

  Future<http.MultipartFile> toMultipartFile({
    required String fieldName,
    required XFile image,
  }) async {
    final bytes = await image.readAsBytes();

    return http.MultipartFile.fromBytes(
      fieldName,
      bytes,
      filename: image.name,
    );
  }
}
```

Figura 9: Adaptador encargado de convertir imágenes `XFile` a `MultipartFile`.

9.3. Para qué lo usé

Lo usé para que la lógica de conversión de imágenes no estuviera repetida dentro del servicio de películas.

Esto fue útil porque mi app permite subir portadas de películas a Cloudinary mediante la API.

10. Singleton Pattern

10.1. Cómo se usa de manera general

El patrón Singleton permite que una clase tenga una sola instancia compartida en toda la aplicación.

Se utiliza cuando se necesita manejar información global, como una sesión, una configuración o un token.

10.2. Cómo lo usé en mi proyecto

En el frontend lo usé en:

```
lib/core/security/session_manager.dart
```

La clase `SessionManager` se encarga de guardar y proporcionar el token de autenticación del usuario.

```

1  /// PATRÓN: Singleton
2  /// Única instancia para persistir token JWT. Accesible desde cualquier capa.
3  class SessionManager {
4      static final SessionManager _instance = SessionManager._internal();
5
6      factory SessionManager() {
7          return _instance;
8      }
9
10     SessionManager._internal();
11
12     String? _token;
13
14     String? get token => _token;
15
16     bool get isAuthenticated => _token != null && _token!.isNotEmpty;
17
18     void saveToken(String token) {
19         _token = token;
20     }
21
22     void clearSession() {
23         _token = null;
24     }
25 }

```

Figura 10: Uso del patrón Singleton para manejar la sesión del usuario mediante `SessionManager`.

10.3. Para qué lo usé

Lo usé para manejar la sesión del usuario desde una sola instancia.

Esto permite que el `ApiClient` pueda tomar el token y enviarlo en las peticiones a la API.

11. DTO / Model

11.1. Cómo se usa de manera general

DTO significa *Data Transfer Object*, que en español se puede entender como Objeto de Transferencia de Datos.

Este patrón se usa para transportar información entre capas o entre el frontend y el backend sin exponer directamente toda la estructura interna del sistema.

11.2. Cómo lo usé en mi proyecto

En el frontend lo usé en modelos como:

```
Pelicula  
ReaccionPelicula  
ResumenReaccion
```

Estos modelos representan los datos que llegan desde la API.

```

// PATRÓN DTO:
// Este objeto representa una película dentro de una colección de reacción.
// Ejemplo: todas las películas guardadas en "Me encanta".
public class PeliculaEnReaccionResponse {

    private Long id;
    private String nombre;
    private String portada;
    private String genero;
    private Double calificacion;
    private Boolean vista;
    private String comentarioPersonal;

    public PeliculaEnReaccionResponse() {
    }

    public PeliculaEnReaccionResponse(Long id, String nombre, String portada, String genero,
                                       Double calificacion, Boolean vista, String comentarioPersonal) {
        this.id = id;
        this.nombre = nombre;
        this.portada = portada;
        this.genero = genero;
        this.calificacion = calificacion;
        this.vista = vista;
        this.comentarioPersonal = comentarioPersonal;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getPortada() {
        return portada;
    }

    public String getGenero() {
        return genero;
    }

    public Double getCalificacion() {
        return calificacion;
    }

    public Boolean getVista() {
        return vista;
    }

    public String getComentarioPersonal() {
        return comentarioPersonal;
    }
}

```

Figura 11: Modelo utilizado para representar y transportar información dentro del frontend.

```
// PATRÓN DTO:
// Este objeto se usa para enviar al frontend una reacción seleccionada,
// mostrando tanto el valor interno como el nombre visible.
public class PeliculaReaccionResponse {

    private String tipoReaccion;
    private String nombre;

    public PeliculaReaccionResponse() {
    }

    public PeliculaReaccionResponse(String tipoReaccion, String nombre) {
        this.tipoReaccion = tipoReaccion;
        this.nombre = nombre;
    }

    public String getTipoReaccion() {
        return tipoReaccion;
    }

    public void setTipoReaccion(String tipoReaccion) {
        this.tipoReaccion = tipoReaccion;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Figura 12: DTO utilizado para transportar información de respuesta entre la API y el frontend.

```

// PATRÓN DTO:
// Este objeto representa el resumen estadístico de una reacción.
// Se usará en la pantalla "Mis reacciones".
public class ReaccionResumenResponse {

    private String tipoReaccion;
    private String nombre;
    private Long total;

    public ReaccionResumenResponse() {
    }

    public ReaccionResumenResponse(String tipoReaccion, String nombre, Long total) {
        this.tipoReaccion = tipoReaccion;
        this.nombre = nombre;
        this.total = total;
    }

    public String getTipoReaccion() {
        return tipoReaccion;
    }

    public void setTipoReaccion(String tipoReaccion) {
        this.tipoReaccion = tipoReaccion;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Long getTotal() {
        return total;
    }

    public void setTotal(Long total) {
        this.total = total;
    }
}

```

Figura 13: DTO utilizado para estructurar información relacionada con reacciones de películas.

En la API lo usé en archivos como:

```

ComentarioPersonalRequest.java
PeliculaReaccionResponse.java
ReaccionResumenResponse.java
PeliculaEnReaccionResponse.java
LoginRequest.java
SignupRequest.java

```

JwtResponse.java

MessageResponse.java

```
// DTO de Pelicula
// Este objeto representa una película dentro de una colección de reacción.
// Ejemplo: todas las películas guardadas en "Me encanta".
public class PeliculaEnReaccionResponse {

    private Long id;
    private String nombre;
    private String portada;
    private String genero;
    private Double calificacion;
    private Boolean vista;
    private String comentarioPersonal;

    public PeliculaEnReaccionResponse() {
    }

    public PeliculaEnReaccionResponse(Long id, String nombre, String portada, String genero,
                                       Double calificacion, Boolean vista, String comentarioPersonal) {
        this.id = id;
        this.nombre = nombre;
        this.portada = portada;
        this.genero = genero;
        this.calificacion = calificacion;
        this.vista = vista;
        this.comentarioPersonal = comentarioPersonal;
    }

    public Long getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getPortada() {
        return portada;
    }

    public String getGenero() {
        return genero;
    }

    public Double getCalificacion() {
        return calificacion;
    }

    public Boolean getVista() {
        return vista;
    }

    public String getComentarioPersonal() {
        return comentarioPersonal;
    }
}
```

Figura 14: DTO de la API utilizado para organizar datos enviados o recibidos por el sistema.

```

// PATRÓN DTO - DATA TRANSFER OBJECT:
// Transporta las credenciales del usuario desde el frontend.
// Valida que usuario y contraseña no estén vacíos antes de procesarse.
import jakarta.validation.constraints.NotBlank;

public class LoginRequest {

    @NotBlank
    private String username;

    @NotBlank
    private String password;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}

```

Figura 15: DTO de autenticación o respuesta utilizado para controlar los datos que salen de la API.

11.3. Para qué lo usé

Lo usé para controlar qué información entra y sale del sistema.

Por ejemplo, para actualizar un comentario personal no necesito mandar toda la película, sino solo el comentario. También para la pantalla de “Mis reacciones” se envía un resumen con el tipo de reacción, el nombre visible y el total.

12. MVC

12.1. Cómo se usa de manera general

MVC significa Modelo Vista Controlador. Es un patrón que separa la aplicación en partes.

El modelo representa los datos, la vista muestra la información al usuario y el controlador recibe solicitudes y coordina la respuesta.

En una API, la vista no siempre aparece igual que en una aplicación visual, pero sí se mantiene la idea de separar controladores, modelos y lógica de datos.

12.2. Cómo lo usé en mi proyecto

En la API Spring Boot se usó mediante la separación de:

```
controllers  
models  
repository  
service
```

Los controladores están en archivos como:

```
PeliculaController.java  
PeliculaReaccionController.java  
AuthController.java
```

```

// PATRÓN MVC - CONTROLLER:
// Recibe las solicitudes HTTP del frontend para gestionar películas.
// Delega la lógica de negocio al servicio y repositorio correspondiente.
import com.example.demo.models.Pelicula;
import com.example.demo.models.User;
import com.example.demo.repository.PeliculaRepository;
import com.example.demo.repository.UserRepository;
import com.example.demo.service.CloudinaryService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/peliculas")
@CrossOrigin(origins = "**")
public class PeliculaController {

    private final PeliculaRepository peliculaRepository;
    private final CloudinaryService cloudinaryService;
    private final UserRepository userRepository;

    public PeliculaController(PeliculaRepository peliculaRepository,
                              CloudinaryService cloudinaryService,
                              UserRepository userRepository) {
        this.peliculaRepository = peliculaRepository;
        this.cloudinaryService = cloudinaryService;
        this.userRepository = userRepository;
    }

    private User obtenerUsuarioAutenticado(Authentication authentication) {
        String username = authentication.getName();
        return userRepository.findByUsername(username)
            .orElseThrow(() -> new RuntimeException("Usuario autenticado no encontrado"));
    }

    private boolean esAdmin(Authentication authentication) {
        return authentication.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
    }

    @PostMapping(consumes = {"multipart/form-data"})
    public ResponseEntity<?> crearPelicula(
        @RequestParam("nombre") String nombre,
        @RequestParam("genero") String genero,
        @RequestParam("calificacion") Double calificacion,
        @RequestParam("vista") boolean vista,
        @RequestParam("imagen") MultipartFile imagen,
        Authentication authentication
    ) {

```

Figura 16: Controlador de la API encargado de recibir peticiones HTTP relacionadas con películas.


```

// Este controlador recibe las solicitudes HTTP del frontend y delega la lógica al servicio.
// No contiene reglas de negocio directamente; solo coordina entrada y salida de datos.
@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "")
public class PeliculaReaccionController {

    private final PeliculaReaccionService peliculaReaccionService;

    public PeliculaReaccionController(PeliculaReaccionService peliculaReaccionService) {
        this.peliculaReaccionService = peliculaReaccionService;
    }

    @PatchMapping("/peliculas/{id}/comentario")
    public ResponseEntity<?> actualizarComentarioPersonal(
        @PathVariable Long id,
        @RequestBody ComentarioPersonalRequest request,
        Authentication authentication
    ) {
        try {
            return ResponseEntity.ok(
                peliculaReaccionService.actualizarComentarioPersonal(id, request, authentication)
            );
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }

    @GetMapping("/peliculas/{id}/reacciones")
    public ResponseEntity<?> obtenerReaccionesDePelicula(
        @PathVariable Long id,
        Authentication authentication
    ) {
        try {
            return ResponseEntity.ok(
                peliculaReaccionService.obtenerReaccionesDePelicula(id, authentication)
            );
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }

    @PostMapping("/peliculas/{id}/reacciones/{tipoReaccion}")
    public ResponseEntity<?> agregarReaccion(
        @PathVariable Long id,
        @PathVariable String tipoReaccion,
        Authentication authentication
    ) {
        try {
            return ResponseEntity.ok(
                peliculaReaccionService.agregarReaccion(id, tipoReaccion, authentication)
            );
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}

```

Figura 17: Controlador de la API encargado de coordinar operaciones del backend.

Los modelos están en:

```

Pelicula.java
PeliculaReaccion.java
User.java
Role.java

```

```

// PATRÓN MVC - CONTROLLER:
// Recibe las solicitudes HTTP del frontend y delega la lógica de autenticación al servicio.
// Evita que el frontend se comunique directamente con la base de datos.
import com.example.demo.models.ERole;
import com.example.demo.models.Role;
import com.example.demo.models.User;
import com.example.demo.payload.request.LoginRequest;
import com.example.demo.payload.request.SignupRequest;
import com.example.demo.payload.response.JwtResponse;
import com.example.demo.payload.response.MessageResponse;
import com.example.demo.repository.RoleRepository;
import com.example.demo.repository.UserRepository;
import com.example.demo.security.jwt.JwtUtils;
import com.example.demo.security.services.UserDetailsImpl;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.*;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;

import java.util.*;
import java.util.stream.Collectors;

@CrossOrigin(origins = "*", maxAge = 3600)
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    AuthenticationManager authenticationManager;

    @Autowired
    UserRepository userRepository;

    @Autowired
    RoleRepository roleRepository;

    @Autowired
    PasswordEncoder encoder;

    @Autowired
    JwtUtils jwtUtils;

    @PostMapping("/signin")
    public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest loginRequest) {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                loginRequest.getUsername(),
                loginRequest.getPassword()
            )
        );
    }

```

Figura 18: Estructura del backend donde se observa la separación entre controladores, modelos, repositorios y servicios.

12.3. Para qué lo usé

Lo usé para que la API estuviera organizada por responsabilidades.

Los controladores reciben las peticiones HTTP, los modelos representan las tablas o entidades y los servicios/repositorios se encargan de la lógica y acceso a datos.

13. Security Filter

13.1. Cómo se usa de manera general

Un Security Filter, o Filtro de Seguridad, intercepta las peticiones antes de que lleguen a los controladores.

Se usa para revisar si una petición tiene permiso para continuar.

13.2. Cómo lo usé en mi proyecto

En la API lo usé en:

```
AuthTokenFilter.java
```

Este filtro valida el token JWT.

JWT significa *JSON Web Token*, que es un token usado para identificar al usuario autenticado.

```

// PATRÓN / COMPONENTE: SECURITY FILTER:
// Intercepta las solicitudes HTTP para validar el token JWT antes de que accedan a los endpoints protegidos.
// Si el token es válido, autenticará al usuario en el contexto de seguridad de Spring.
import com.example.demo.security.services.UserDetailsServiceImpl;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.slf4j.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class AuthTokenFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtils jwtUtils;

    @Autowired
    private UserDetailsServiceImpl userDetailsService;

    private static final Logger logger = LoggerFactory.getLogger(AuthTokenFilter.class);

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {
        try {
            String jwt = parseJwt(request);

            if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
                String username = jwtUtils.getUserNameFromJwtToken(jwt);

                UserDetails userDetails = userDetailsService.loadUserByUsername(username);
                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                        userDetails,
                        null,
                        userDetails.getAuthorities()
                    );

                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        } catch (Exception e) {
            logger.error("No se pudo establecer la autenticación del usuario: {}", e.getMessage());
        }
    }
}

```

Figura 19: Filtro de seguridad AuthTokenFilter.java, encargado de validar el token JWT antes de permitir el acceso a rutas protegidas.

13.3. Para qué lo usé

Lo usé para proteger las rutas de la API.

Antes de que un usuario pueda crear, editar, eliminar o consultar películas, el sistema revisa si tiene un token válido.

14. Conclusión

En conclusión, aplicar patrones de diseño y principios SOLID en la construcción del proyecto me ayudó a tener un código más organizado, reutilizable y fácil de mantener.

En este proyecto aprendí que los patrones de diseño no deben usarse solo por cumplir un requisito, sino porque ayudan a separar responsabilidades dentro del sistema. También comprendí que SOLID no es un conjunto de patrones, sino una guía de principios para escribir código más claro y menos acoplado.